8 Ways to Scale Any System

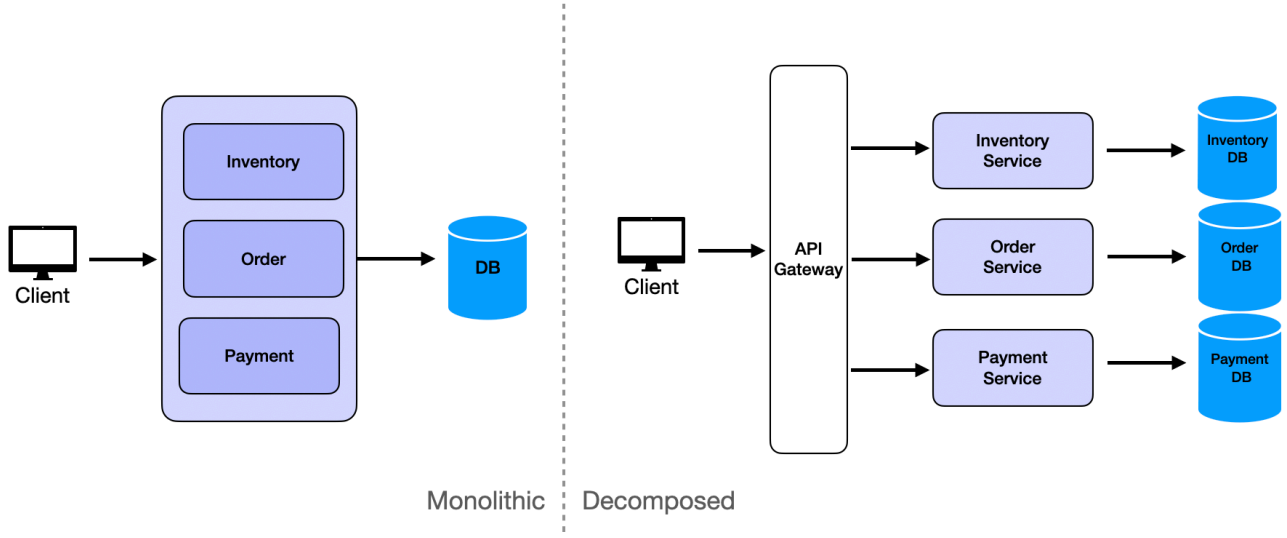
systemdesignschool.io/fundamentals/how-to-scale-a-system

How to Scale a System

Scaling a system effectively is one of the most critical aspects of satisfying non-functional requirements in system design. Scalability, in particular, is often a top priority. Below, we explore various strategies to achieve scalable system architecture.

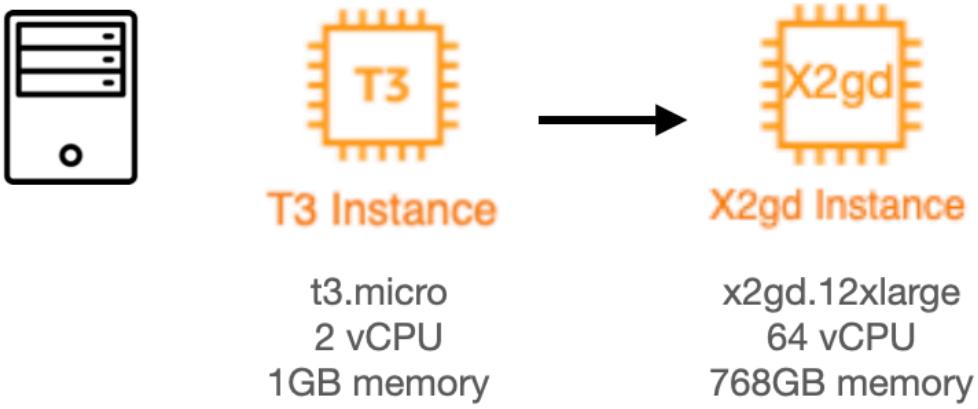
Decomposition

Decomposition involves breaking down requirements into microservices. The key principle is to divide the system into smaller, independent services based on specific business capabilities or requirements. Each microservice should focus on a single responsibility to enhance scalability and maintainability.



Vertical Scaling

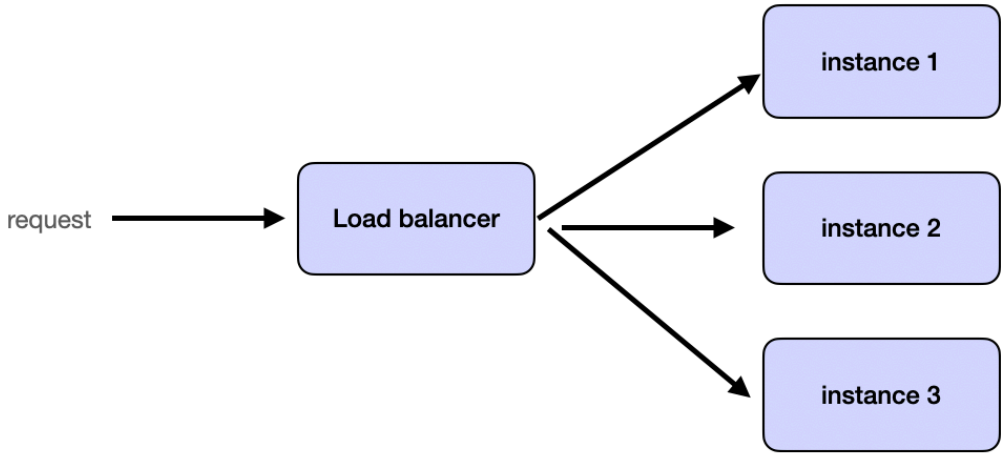
Vertical scaling represents the brute force approach to scaling. The concept is straightforward: scale up by using more powerful machines. Thanks to advancements in cloud computing, this approach has become much more feasible. While in the past, organizations had to wait for new machines to be built and shipped, today they can spin up new instances in seconds.



Modern cloud providers offer impressive vertical scaling options. For instance, AWS provides "Amazon EC2 High Memory" instances with up to 24 TB of memory, while Google Cloud offers "Tau T2D" instances specifically optimized for compute-intensive workloads.

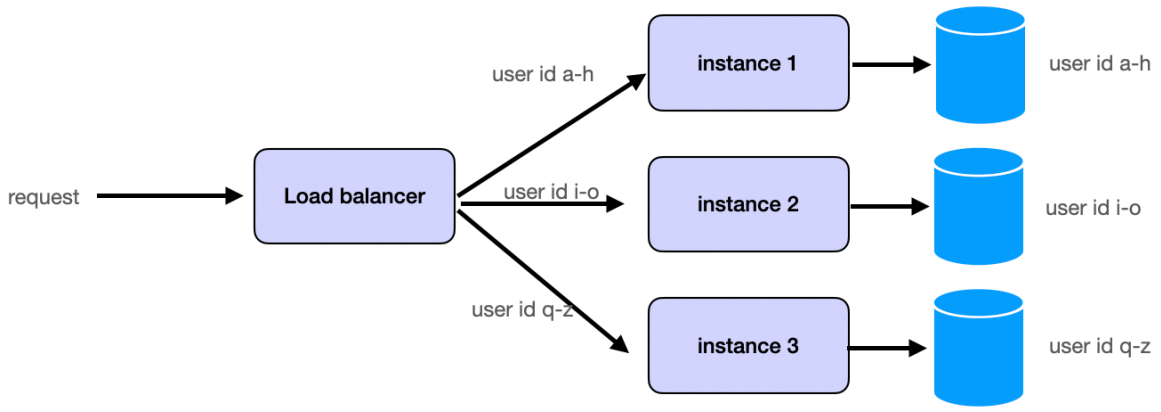
Horizontal Scaling

Horizontal scaling focuses on scaling out by running **multiple identical instances** of stateless services. The stateless nature of these services enables seamless distribution of requests across instances using load balancers.



Partitioning

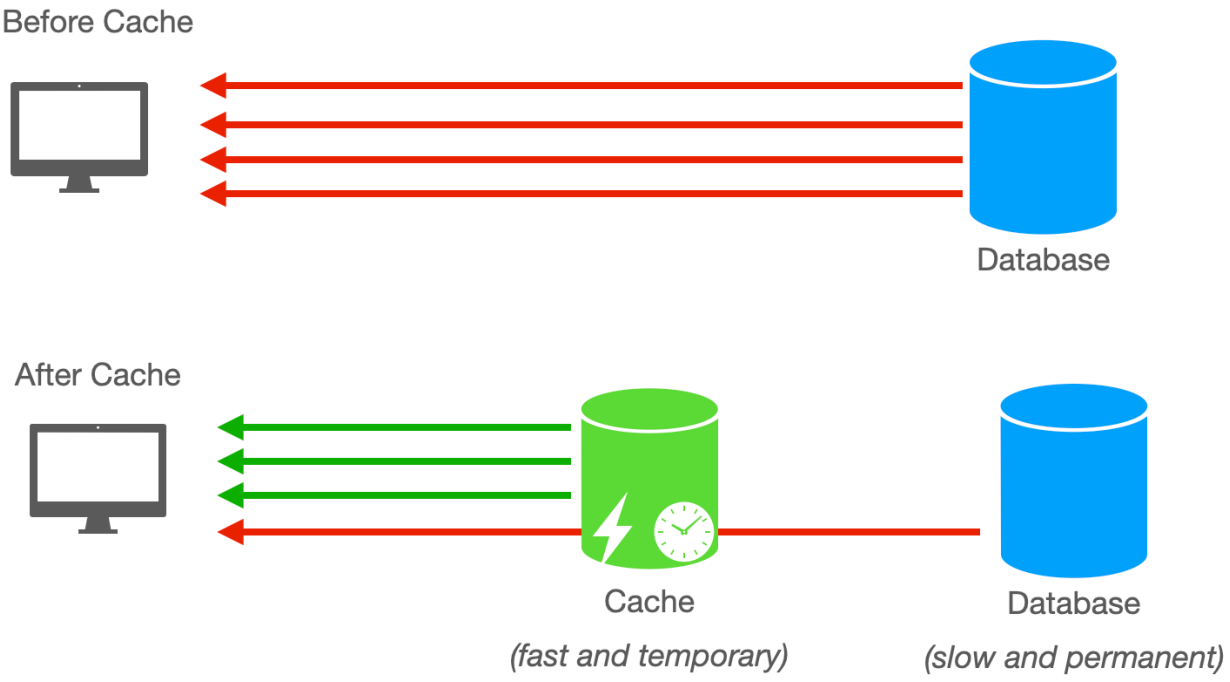
Partitioning involves **splitting requests and data into shards** and distributing them across services or databases. This can be accomplished by partitioning data based on user ID, geographical location, or another logical key. Many systems implement consistent hashing to ensure balanced partitioning.



We will cover partitioning in great detail in the Database Partitioning section.

Caching

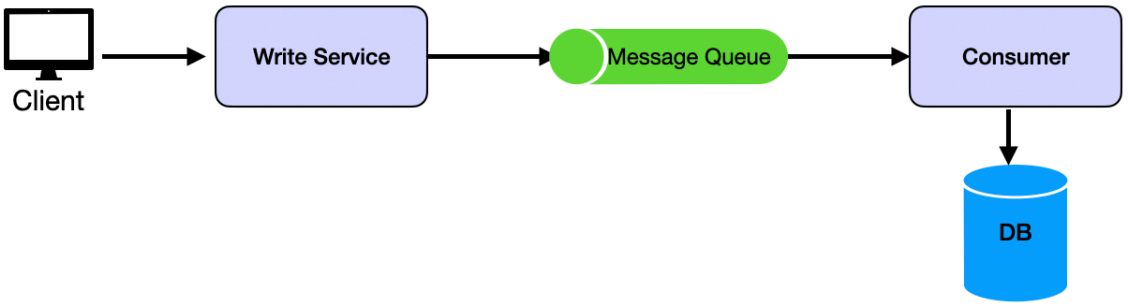
Caching serves to improve query read performance by storing frequently accessed data in faster memory storage, such as in-memory caches. Popular tools like Redis or Memcached can effectively store hot data to reduce database load.



We will cover caching in great detail in the Caching section.

Buffer with Message Queues

High-concurrency scenarios often encounter write-intensive operations. Frequent database writes can overload the system due to disk I/O bottlenecks. Message queues can buffer write requests, changing synchronous operations into asynchronous ones, thereby limiting database write requests to manageable levels and preventing system crashes.

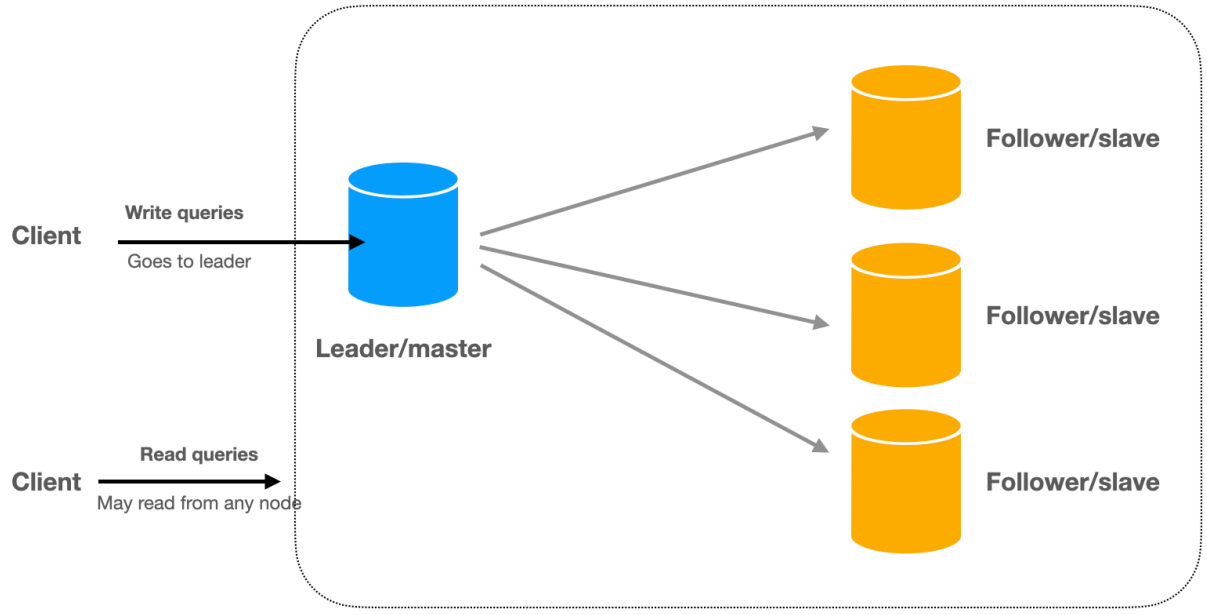


We will cover message queues in great detail in the Message Queues section.

Separating Read and Write

Whether a system is read-heavy or write-heavy depends on the business requirements. For example, a social media platform is read-heavy because users read more than they write. On the other hand, an IOT system is write-heavy because users write more than they read. This is why we want to separate read and write operations to treat them differently.

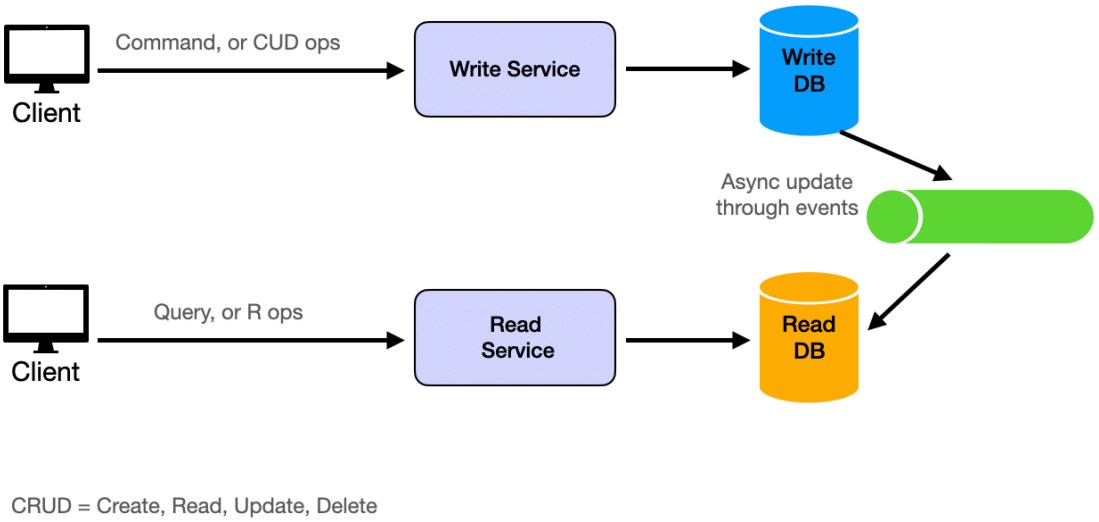
Read and write separation typically involves two main strategies. First, replication implements a **leader-follower architecture** where writes occur on the leader, and **followers provide read replicas**.



Second, the so-called **CQRS (Command Query Responsibility Segregation) pattern** takes read-write separation further by using completely different models for reading and writing data. In CQRS, the system is split into two parts:

- **Command Side (write side)**: Handles all write operations (create, update, delete) using a data model optimized for writes
- **Query Side (read side)**: Handles all read operations using a denormalized data model optimized for reads

Changes from the command side are **asynchronously propagated to the query side**.



CRUD = Create, Read, Update, Delete

For example, a system might use MySQL as the source-of-truth database while employing Elasticsearch for full-text search or analytical queries, and asynchronously sync changes from MySQL to Elasticsearch using MySQL binlog **Change Data Capture (CDC)**.

Combining Techniques

Effective scaling usually requires a multi-faceted approach combining several techniques. This starts with decomposition to break down monolithic services for independent scaling. Then, partitioning and caching work together to distribute load efficiently while enhancing performance. Read/write separation ensures fast reads and reliable writes through leader-replica setups. Finally, business logic adjustments help design strategies that mitigate operational bottlenecks without compromising user experience.

Adapting to Changing Business Requirements

Adapting business requirements offers a practical way to handle large traffic loads. While not strictly a technical approach, understanding these strategies demonstrates valuable experience and critical thinking skills in an interview setting.

Consider a weekly sales event scenario: Instead of running all sales simultaneously for all users, the load can be distributed by allocating specific days for different product categories for specific regions. For instance, baby products might be featured on Day 1, followed by electronics on Day 2. This approach ensures more predictable traffic patterns and enables better resource allocation such as pre-loading the cache for the upcoming day and scaling out the read replicas for the specific regions.

Another example involves handling consistency challenges during high-stakes events like eBay auctions. By temporarily displaying bid success messages on the frontend, the system can provide a seamless user experience while the backend resolves consistency issues asynchronously. Users will eventually see the correct status of their bid after the auction ends.

While these are not technical solutions, bringing them up in an interview demonstrates your ability to think through the problem and provide practical solutions.